

Story Consistency Checker - Task 5

What You're Building

Tasks 1–4 gave you the foundation: tokens, vectors, chunking, a vector database, and a working `verify_claim()` pipeline.

Task 5 is where the project becomes a real product.

You will measure how good your system actually is, make it smarter with Chain-of-Thought prompting, improve retrieval with Hybrid Search, add autonomous agent behaviour, and wrap the whole thing in a user-facing UI for your final demo.

Pipeline at the end of Task 5:
Claim → Hybrid Retrieval → CoT Reasoning → Agent Loop → Streamlit UI → Verdict

Task 5 Breakdown at a Glance

Part	Topic	Core Concept	Deliverable
Part 1	Evaluation	<i>You can't improve what you don't measure</i>	Accuracy report
Part 2	Chain-of-Thought Prompting	<i>Force the model to show its work</i>	CoT vs standard comparison
Part 3	Hybrid Search	<i>Vectors + keywords = better retrieval</i>	Updated pipeline
Part 4	Agents & Tool Use	<i>The pipeline thinks for itself</i>	Agent trace log
Part 5	Streamlit UI	<i>Make it usable for non-coders</i>	Working web app

Part 1 : Evaluation

Running your Task 4 MVP on one or two examples tells you very little about how reliable your system really is. A proper evaluation requires a structured test dataset with diverse case types and a rigorous process for diagnosing failures. This is the foundation that makes every future improvement meaningful.

Your Task

1. Run your Task 4 pipeline against the provided test.csv dataset from the Backstory folder.
2. Record the predicted label for each row and compare against the ground-truth label column.
3. Compute overall accuracy, and break it down by case type: easy consistent, easy inconsistent, and hard cases (subtle contradictions, paraphrasing, negation, numbers).
4. For each failure, diagnose the root cause:
 - Retrieval failure - the wrong chunks were retrieved, so the model never saw the relevant text.
 - Reasoning failure - the right chunks were retrieved, but the model drew the wrong conclusion.

Tip: To distinguish retrieval vs reasoning failures, print the top-3 retrieved chunks for each failing case. If the answer is there but the model got it wrong, it's a reasoning failure. If the answer is absent from the chunks, it's a retrieval failure.

Deliverable

An accuracy report that includes:

- Overall accuracy score (correct / total)
- Accuracy broken down by case type (easy consistent, easy inconsistent, hard)
- A table of all failing cases, each annotated with the failure type (retrieval or reasoning)
- A brief written analysis: what surprised you, and where does your system struggle most?

Resources

- RAGAS: Evaluation Framework for RAG - <https://docs.ragas.io/en/stable/>
- Precision, Recall, and Accuracy as metrics for retrieval systems

Part 2 - Prompt Engineering: Chain-of-Thought

Concept: Standard Prompts Fail on Implicit Details

A standard prompt asks the model: “Is this claim consistent with the story? Answer 1 or 0.” This works for simple, explicit facts. It breaks down on paraphrased information, negation, numerical comparisons, or anything requiring multi-step inference.

Chain-of-Thought (CoT) prompting forces the model to reason step-by-step before committing to a verdict. Because the model must articulate what the story says, what the claim says, and how they relate, it reads the context more carefully and makes fewer logical errors.

Your Task

5. Implement a `build_cot_prompt(claim, chunks)` function with explicit reasoning steps embedded in the prompt.
6. Re-run your full evaluation dataset (from Part 1) using the CoT prompt instead of the standard prompt.
7. Compare results side by side: standard accuracy vs. CoT accuracy, broken down by case type.
8. Identify at least one concrete example where the standard prompt gave the wrong answer but the CoT prompt got it right. Explain in your own words why the step-by-step structure helped.

Deliverable

- A `build_cot_prompt()` function in your notebook
- A side-by-side accuracy table: Standard vs. CoT, overall and by case type
- One annotated example showing the standard prompt’s wrong reasoning vs. the CoT prompt’s correct reasoning, with your written explanation

Resources

- Chain-of-Thought Prompting Elicits Reasoning in Large Language Models - <https://arxiv.org/pdf/2201.11903>
- Zero-Shot CoT Guide – <https://medium.com/@gangelin/lets-think-step-by-step-zero-shot-chain-of-thought-zero-shot-cot-reasoning-in-large-language-2dfd21315d19>

Part 3 - Advanced Retrieval: Hybrid Search

Concept: Semantic Search Is Blind to Exact Matches

Your current vector search finds chunks that are semantically similar to a claim. This works well for paraphrased facts but fails on exact names, dates, and specific numbers - because two texts can be semantically distant even when one is literally contained in the other.

Keyword search (BM25) is the opposite: it excels at exact token matches but has no concept of meaning. Hybrid Search combines both, so you get the best of each approach. This is the retrieval strategy used by production RAG systems.

Method	Strength	Weakness
Vector Search	Understands paraphrasing and semantics	Misses exact names, dates, numbers
BM25 Keyword Search	Precise on exact tokens	No understanding of meaning
Hybrid Search	Both semantic and exact match coverage	Slightly more complex to implement

Your Task

9. Implement a BM25 keyword retriever alongside your existing vector retriever (use the `rank_bm25` library or LangChain's `BM25Retriever`).
10. Merge the results from both retrievers into a single ranked list for each claim. A simple approach is Reciprocal Rank Fusion (RRF): score each chunk by $1/(\text{rank} + k)$ from each list, then sum and sort.
11. (Optional but recommended) Implement re-ranking: retrieve the top 10 hybrid results, then use a Cross-Encoder to score and keep the best 3.
12. Re-run your evaluation dataset with the hybrid retriever and compare retrieval failure rates against your Part 1 baseline.

Deliverable

- Updated pipeline code using Hybrid Search
- A retrieval comparison: how many retrieval failures from Part 1 did Hybrid Search fix?
- (Optional) Re-ranking implementation and its effect on accuracy

Resources

- Pinecone: Hybrid Search Explained - <https://docs.pinecone.io/guides/search/hybrid-search>
- `rank_bm25` Python library
- Cohere Rerank Endpoint (for understanding re-ranking) - <https://cohere.com/blog/rerank-4>

Part 4 : Agents & Tool Use

Concept: The Pipeline Should Think For Itself

Your current system is a linear pipeline: embed claim → retrieve → prompt → verdict. It always follows the same steps regardless of how confident it is. A real agent can reflect on its own output and decide to take additional steps when needed.

You will build a simple reasoning loop: if the model reads the retrieved chunks and determines it does not have enough information to give a reliable verdict, it triggers a new, refined search query instead of guessing.

Your Task

13. Implement an agent loop using one of the following approaches: LangGraph, raw function calling with your LLM, or a simple custom loop with a “need_more_info” flag in the model’s output.
14. Define a tool the agent can call: `search_story(query)` - which runs a hybrid retrieval and returns chunks.
15. The agent’s first action is always to search for the claim. After reading, if it decides it needs more context, it formulates a new query and calls `search_story()` again before giving a final verdict.
16. Produce a log trace for at least one multi-hop example: a claim that required two searches to verify correctly.

Deliverable

- Agent implementation in your notebook
- A log trace showing a multi-hop reasoning sequence: Search A → Read → Decision to Search B → Final Verdict
- A brief note on which types of claims benefited most from the agent loop

Resources

- ReAct: Synergizing Reasoning and Acting in Language Models — [arxiv.org](https://arxiv.org/abs/2210.03629)
- DeepLearning.AI: AI Agents in LangGraph
- What are AI Agents? (IBM Technology)

Part 5 - User Interface & Final Demo

Concept: Make It Usable For Non-Coders

A Jupyter notebook is a great development environment, but it is not a product. In this final part, you wrap your entire pipeline in a Streamlit web application that any user can interact with, upload a story, enter a claim, and get a verdict - without ever touching code.

This is your final demo. Polish matters. The UI should be clear, responsive, and show the model's reasoning process so the user understands how the verdict was reached.

Your Task

17. Create a Streamlit app (app.py) that wraps your complete pipeline: vector DB setup, hybrid retrieval, CoT prompting, agent loop, and verdict.
18. The app must include the following features:
 - File Uploader - the user uploads a plain-text story file (.txt). The app chunks it, embeds it, and builds the vector store on the fly.
 - Claim Input - a text box where the user types a claim to verify.
 - Status Bar / Thinking Panel - a live display showing what the agent is doing (e.g. "Searching story...", "Retrieved 3 chunks...", "Reasoning...").
 - Verdict Display - a clear, prominent output: Consistent or Inconsistent, with the CoT reasoning shown below.
19. Run your app locally and prepare a live demo. Walk through at least one consistent claim and one inconsistent claim during your presentation.

Deliverable

- A working Streamlit app (app.py) that runs end-to-end
- A live demo during the final session: upload a story, enter claims, show verdicts and reasoning
- A brief reflection: what would you improve if you had another week?

Resources

- Streamlit: Build an LLM App - <https://docs.streamlit.io/develop/tutorials/chat-and-llm-apps/llm-quickstart>
- Streamlit `st.status()` and `st.spinner()` for the thinking panel

Dataset & Resources Reminder

You are continuing to use the same dataset from Tasks 1–4.

Books	Two complete story text files - your vector database source
train.csv	Backstory folder - use for building and debugging your system
test.csv	Backstory folder - use for all evaluation in Part 1 and onward
content column	The actual claim to feed into your pipeline
label column	Ground truth: 1 = consistent, 0 = inconsistent
Drive link	Same folder shared in Task 4 (books + Backstory)

Colab Notebook

Continue working in your Task 4 Colab notebook. Add new sections for each Part - do not start a new notebook. Keep your Task 4 setup cells intact so your vector DB and pipeline are available to all new parts.

You are free to use any LLM: OpenAI, Anthropic Claude, or a free local model via HuggingFace. All options remain valid.

What the Submission Looks Like

Part	Strong Submission	Common Mistake
Part 1: Evaluation	Detailed failure analysis per case, retrieval vs reasoning distinction clearly made	Only reporting overall accuracy with no failure breakdown
Part 2: CoT	Concrete example with annotated wrong vs right reasoning, written explanation of why CoT helped	Just showing accuracy improved without explaining the mechanism
Part 3: Hybrid Search	Retrieval failure count before and after, optional re-ranking implemented	Implementing BM25 but not comparing against the Part 1 baseline
Part 4: Agents	Clear log trace with two search steps, JSON-structured agent output	A linear pipeline relabelled as an agent with no actual decision loop
Part 5: UI	All three features working, live demo with both claim types, a genuine reflection	A UI that crashes on upload or has no

		status/reasoning display
--	--	--------------------------

Notes

- All five parts feed into each other. Complete them in order - Part 2 accuracy numbers depend on Part 1 baselines, Part 3 fixes Part 1 retrieval failures, and so on.
- Your evaluation dataset quality matters. Weak test cases give misleadingly high accuracy. Push yourself on the hard cases.
- There is no single correct Streamlit design. Focus on clarity: the user should always understand what the app is doing and why it returned the verdict it did.
- If you get stuck on the agent loop, start with the simplest version: a while loop that checks a flag, not a full framework. Get it working, then refine.

This is the final task. By the end, you will have built a complete, evaluated, agent-powered, UI-wrapped Story Consistency Checker - a system you can demonstrate and be proud of.

Good luck ! build something great.